

UNIT- I

UNIT-I

Introduction:-Purpose of testing,Dichotomies,model for testing,consequences of bugs, taxonomy of bugs,Flow graphs and Path testing:- Basics concepts of path testing, predicates, path predicates and achievable paths, path sensitizing, path instrumentation, application of path testing.

Testing:

Testing is the process of exercising or evaluating a system or system components by manual or automated means to verify that it satisfies specified requirements.

The Purpose of Testing

Testing consumes at least half of the time and work required to produce a functional program.

MYTH: Good programmers write code without bugs. (It's wrong!!!) . History says that even well written programs still have 1-3 bugs per hundred statements.

Productivity and Quality in Software:

- In production of consumer goods and other products, every manufacturing stage is subjected to quality control and testing from component to final stage.
- If flaws are discovered at any stage, the product is either discarded or cycled back for rework and correction.
- Productivity is measured by the sum of the costs of the material, the rework, and the discarded components, and the cost of quality assurance and testing.
- There is a tradeoff between quality assurance costs and manufacturing costs: If sufficient time is not spent in quality assurance, the reject rate will be high and so will be the net cost. If inspection is good and all errors are caught as they occur, inspection costs will dominate, and again the net cost will suffer.
- Testing and Quality assurance costs for 'manufactured' items can be as low as 2% in consumer products or as high as 80% in products such as space-ships, nuclear reactors, and aircrafts, where failures threaten life. Whereas the manufacturing cost of software is trivial.
- The biggest part of software cost is the cost of bugs: the cost of detecting them, the cost of correcting them, the cost of designing tests that discover them, and the cost of running those tests.
- For software, quality and productivity are indistinguishable because the cost of a software copy is trivial.
- Testing and Test Design are parts of quality assurance should also focus on bug prevention. A prevented bug is better than a detected and corrected bug.

Phases in a tester's mental life:

Phases in a tester's mental life can be categorized into the following 5 phases:

1. **Phase 0: (Until 1956: Debugging Oriented)** There is no difference between testing and debugging. Phase 0 thinking was the norm in early days of software development till testing emerged as a discipline.
2. **Phase 1: (1957-1978: Demonstration Oriented)** the purpose of testing here is to show that software works. Highlighted during the late 1970s. This failed because the probability of showing that software works 'decreases' as testing increases. I.e. the more you test, the more likely you will find a bug.
3. **Phase 2: (1979-1982: Destruction Oriented)** the purpose of testing is to show that software doesn't work. This also failed because the software will never get released as you will find one bug or the other. Also, a bug corrected may also lead to another bug.
4. **Phase 3: (1983-1987: Evaluation Oriented)** the purpose of testing is not to prove anything but to reduce the perceived risk of not working to an acceptable value (Statistical Quality Control). Notion is that testing does improve the product to the extent that testing catches bugs and to the extent that those bugs are fixed. The product is released when the confidence on that product is high enough. (Note: This is applied to large software products with millions of code and years of use.)
5. **Phase 4: (1988-2000: Prevention Oriented)** Testability is the factor considered here. One reason is to reduce the labor of testing. Other reason is to check the testable and nontestable code. Testable code has fewer bugs than the code that's hard to test. Identifying the testing techniques to test the code is the main key here.

Test Design:

We know that the software code must be designed and tested, but many appear to be unaware that tests themselves must be designed and tested. Tests should be properly designed and tested before applying it to the actual code.

Testing isn't everything:

There are approaches other than testing to create better software. Methods other than testing include:

1. **Inspection Methods:** Methods like walkthroughs, desk checking, formal inspections and code reading appear to be as effective as testing but the bugs caught don't completely overlap.
2. **Design Style:** While designing the software itself, adopting stylistic objectives such as testability, openness and clarity can do much to prevent bugs.

3. **Static Analysis Methods:** Includes formal analysis of source code during compilation. In earlier days, it is a routine job of the programmer to do that. Now, the compilers have taken over that job.
4. **Languages:** The source language can help reduce certain kinds of bugs. Programmers find new bugs while using new languages.

Development Methodologies and Development Environment: The development process and the environment in which that methodology is embedded can prevent many kinds of bugs.

DICHOTOMIES

Testing Versus Debugging:

Many people consider both as same. Purpose of testing is to show that a program has bugs. The purpose of testing is to find the error or misconception that led to the program's failure and to design and implement the program changes that correct the error.

Debugging usually follows testing, but they differ as to goals, methods and most important psychology. The below table shows few important differences between testing and debugging.

Testing	Debugging
Testing starts with known conditions, uses predefined procedures and has predictable outcomes.	Debugging starts from possibly unknown initial conditions and the end cannot be predicted except statistically.
Testing can and should be planned, designed and scheduled.	Procedure and duration of debugging cannot be so constrained.
Testing is a demonstration of error or apparent correctness.	Debugging is a deductive process.
Testing proves a programmer's failure.	Debugging is the programmer's vindication (Justification).
Testing, as executes, should strive to be predictable, dull, constrained, rigid and inhuman.	Debugging demands intuitive leaps, experimentation and freedom.
Much testing can be done without design knowledge.	Debugging is impossible without detailed design knowledge.
Testing can often be done by an outsider.	Debugging must be done by an insider.
Much of test execution and design can be automated.	Automated debugging is still a dream.

Functional testing can be designed from a functional or a structural point of view.

In, the program or system is treated as a black box. It is subjected to inputs, and its outputs are verified for conformance to specified behavior. Functional testing takes the user point of view- bother about functionality and features and not the program's implementation.

In **Structural testing** does look at the implementation details. Things such as programming style, control method, source language, database design, and coding details dominate structural testing. o Both Structural and functional tests are useful, both have limitations, and both target different kinds of bugs. Functional tests can detect all bugs but would take infinite time to do so. Structural tests are inherently finite but cannot detect all errors even if completely executed.

Designer versus Tester

Test designer is the person who designs the tests where as the tester is the one actually tests the code. During functional testing, the designer and tester are probably different persons. During unit testing, the tester and the programmer merge into one person. Tests designed and executed by the software designers are by nature biased towards structural consideration and therefore suffer the limitations of structural testing.

Modularity versus Efficiency

A module is a discrete, well-defined, small component of a system. Smaller the modules, difficult to integrate; larger the modules, difficult to understand. Both tests and systems can be modular. Testing can and should likewise be organized into modular components. Small, independent test cases can be designed to test independent modules.

Small versus Large

Programming in large means constructing programs that consists of many components written by many different programmers. Programming in the small is what we do for ourselves in the privacy of our own offices. Qualitative and Quantitative changes occur with size and so must testing methods and quality criteria.

Builder versus Buyer

Most software is written and used by the same organization. Unfortunately, this situation is dishonest because it clouds accountability. If there is no separation between builder and buyer, there can be no accountability.

The different roles / users in a system include:

1. **Builder:** Who designs the system and is accountable to the buyer.
2. **Buyer:** Who pays for the system in the hope of profits from providing services?
3. **User:** Ultimate beneficiary or victim of the system. The user's interests are also guarded by.
4. **Tester:** Who is dedicated to the builder's destruction?
5. **Operator:** Who has to live with the builders' mistakes, the buyers' murky (unclear) specifications, testers' oversights and the users' complaints?

MODEL FOR TESTING:

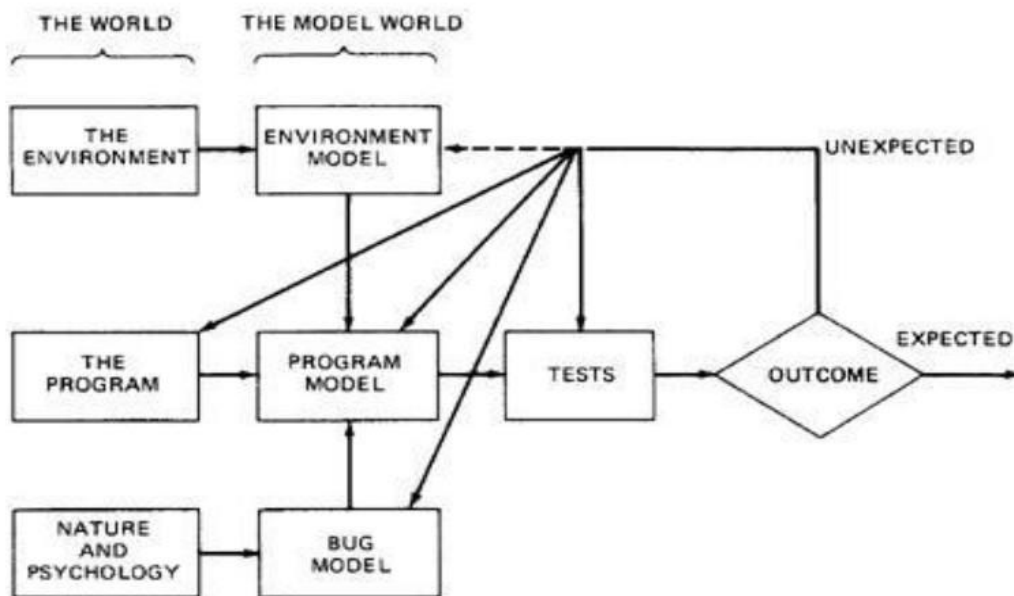


Figure 1.1: A Model for Testing

Above figure is a model of testing process. It includes three models: A model of the environment, a model of the program and a model of the expected bugs.

Environment: A Program's environment is the hardware and software required to make it run. For online systems, the environment may include communication lines, other systems, terminals and operators. o The environment also includes all programs that interact with and are used to create the program under test - such as OS, linkage editor, loader, compiler, utility routines.

Because the hardware and firmware are stable, it is not smart to blame the environment for bugs.

Program: Most programs are too complicated to understand in detail.

The concept of the program is to be simplified in order to test it. If simple model of the program doesn't explain the unexpected behavior, we may have to modify that model to include more facts and details. And if that fails, we may have to modify the program.

Bugs: Bugs are more insidious (deceiving but harmful) than ever we expect them to be.

An unexpected test result may lead us to change our notion of what a bug is and our model of bugs. Some optimistic notions that many programmers or testers have about bugs are usually unable to test effectively and unable to justify the dirty tests most programs need.

Optimistic notions about bugs:

1. **Benign Bug Hypothesis:** The belief that bugs are nice, tame and logical.
(Benign: Not Dangerous)
2. **Bug Locality Hypothesis:** The belief that a bug discovered with in a component affects only that component's behavior.
3. **Control Bug Dominance:** The belief those errors in the control structures (if, switch etc) of programs dominate the bugs.
4. **Code / Data Separation:** The belief that bugs respect the separation of code and data.
5. **Lingua Salvatore Est.:** The belief that the language syntax and semantics (e.g. Structured Coding, Strong typing, etc) eliminates most bugs.
6. **Corrections Abide:** The mistaken belief that a corrected bug remains corrected.
7. **Silver Bullets:** The mistaken belief that X (Language, Design method, representation, environment) grants immunity from bugs.
8. **Sadism Suffices:** The common belief (especially by independent tester) that a sadistic streak, low cunning, and intuition are sufficient to eliminate most bugs. Tough bugs need methodology and techniques.

Angelic Testers: The belief that testers are better at test design than programmers is at code design.

Tests:

Tests are formal procedures, Inputs must be prepared, Outcomes should predict, tests should be documented, commands need to be executed, and results are to be observed. All these errors are subjected to error.

We do three distinct kinds of testing on a typical software system. They are:

1. Unit / Component Testing: A **Unit** is the smallest testable piece of software that can be compiled, assembled, linked, loaded etc. A unit is usually the work of one programmer and consists of several hundred or fewer lines of code. Unit Testing is the testing we do to show that the unit does not satisfy its functional specification or that its implementation structure does not match the intended design structure. A **Component** is an integrated aggregate of one or more units. Component Testing is the testing we do to show that the component does not satisfy its functional specification or that its implementation structure does not match the intended design structure.

2. Integration Testing: **Integration** is the process by which components are aggregated to create larger components. Integration Testing is testing done to show that even though the components were individually satisfactory (after passing component testing), checks the combination of components are incorrect or inconsistent.

3. System Testing: A **System** is a big component. System Testing is aimed at revealing bugs that cannot be attributed to components. It includes testing for performance, security, accountability, configuration sensitivity, startup and recovery.

Role of Models: The art of testing consists of creating, selecting, exploring, and revising models. Our ability to go through this process depends on the number of different models we have at hand and their ability to express a program's behaviour.

CONSEQUENCES OF BUGS

Importance of bugs: The importance of bugs depends on frequency, correction cost, installation cost, and consequences.

1. **Frequency:** How often does that kind of bug occur? Pay more attention to the more frequent bug types.
2. **Correction Cost:** What does it cost to correct the bug after it is found? The cost is the sum of 2 factors: (1) the cost of discovery (2) the cost of correction. These costs go up dramatically later in the development cycle when the bug is discovered. Correction cost also depends on system size.
3. **Installation Cost:** Installation cost depends on the number of installations: small for a single user program but more for distributed systems. Fixing one bug and distributing the fix could exceed the entire system's development cost.
4. **Consequences:** What are the consequences of the bug? Bug consequences can range from mild to catastrophic.

A reasonable metric for bug importance is

Importance= (\$) = Frequency * (Correction cost + Installation cost + Consequential cost)

Consequences of bugs: The consequences of a bug can be measure in terms of human rather than machine. Some consequences of a bug on a scale of one to ten are:

- 1 **Mild:** The symptoms of the bug offend us aesthetically (gently); a misspelled output or a misaligned printout.
- 2 **Moderate:** Outputs are misleading or redundant. The bug impacts the system's performance.
- 3 **Annoying:** The system's behavior because of the bug is dehumanizing.
E.g. Names are truncated or arbitrarily modified.
- 4 **Disturbing:** It refuses to handle legitimate (authorized / legal) transactions. The ATM won't give you money. My credit card is declared invalid.
- 5 **Serious:** It loses track of its transactions. Not just the transaction itself but the fact that the transaction occurred. Accountability is lost.
- 6 **Very Serious:** The bug causes the system to do the wrong transactions. Instead of losing your paycheck, the system credits it to another account or converts deposits to withdrawals.
- 7 **Extreme:** The problems aren't limited to a few users or to few transaction types. They are frequent and arbitrary instead of sporadic infrequent) or for unusual cases.
- 8 **Intolerable:** Long term unrecoverable corruption of the database occurs and the corruption is not easily discovered. Serious consideration is given to shutting the system down.
- 9 **Catastrophic:** The decision to shut down is taken out of our hands because the system fails.
- 10 **Infectious:** What can be worse than a failed system? One that corrupt other systems even though it does not fall in itself ; that erodes the social physical environment; that melts nuclear reactors and starts war.

Flexible severity rather than absolutes

Quality can be measured as a combination of factors, of which number of bugs and their severity is only one component. o Many organizations have designed and used satisfactory, quantitative, quality metrics.

Because bugs and their symptoms play a significant role in such metrics, as testing progresses, you see the quality rise to a reasonable value which is deemed to be safe to ship the product. o The factors involved in bug severity are:

1. **Correction Cost:** Not so important because catastrophic bugs may be corrected easier and small bugs may take major time to debug.

2. **Context and Application Dependency:** Severity depends on the context and the application in which it is used.
3. **Creating Culture Dependency:** What's important depends on the creators of software and their cultural aspirations. Test tool vendors are more sensitive about bugs in their software than games software vendors.
4. **User Culture Dependency:** Severity also depends on user culture. Naive users of PC software go crazy over bugs where as pros (experts) may just ignore.
5. **The software development phase:** Severity depends on development phase. Any bugs gets more severe as it gets closer to field use and more severe the longer it has been around.

TAXONOMY OF BUGS

There is no universally correct way to categorize bugs. The taxonomy is not rigid.

A given bug can be put into one or another category depending on its history and the programmer's state of mind.

The major categories are: (1) Requirements, Features and Functionality Bugs (2) Structural Bugs (3) Data Bugs (4) Coding Bugs (5) Interface, Integration and System Bugs (6) Test and Test Design Bugs.

I Requirements, Features and Functionality Bugs

Various categories in Requirements, Features and Functionality bugs include:

1. Requirements and Specifications Bugs:

- Requirements and specifications developed from them can be incomplete, ambiguous, or self-contradictory. They can be misunderstood or impossible to understand.
- The specifications that don't have flaws in them may change while the design is in progress. The features are added, modified and deleted.
- Requirements, especially, as expressed in specifications are a major source of expensive bugs.
- The range is from a few percentages to more than 50%, depending on the application and environment. What hurts most about the bugs is that they are the earliest to invade the system and the last to leave.

2. Feature Bugs:

- Specification problems usually create corresponding feature problems.
- A feature can be wrong, missing, or superfluous (serving no useful purpose). A missing feature or case is easier to detect and correct. A wrong feature could have deep design implications.
- Removing the features might complicate the software, consume more resources, and foster more bugs.

3. Feature Interaction Bugs:

- Providing correct, clear, implementable and testable feature specifications is not enough.
- Features usually come in groups or related features. The features of each group and the interaction of features within the group are usually well tested.
- The problem is unpredictable interactions between feature groups or even between individual features. For example, your telephone is provided with call holding and call forwarding. The interactions between these two features may have bugs.
- Every application has its peculiar set of features and a much bigger set of unspecified feature interaction potentials and therefore result in feature interaction bugs.

Specification and Feature Bug Remedies:

Most feature bugs are rooted in human-to-human communication problems. One solution is to use high-level, formal specification languages or systems. Such languages and systems provide short term support but in the long run, does not solve the problem.

Short term Support: Specification languages facilitate formalization of requirements and inconsistency and ambiguity analysis.

Long term Support: Assume that we have a great specification language and that can be used to create unambiguous, complete specifications with unambiguous complete tests and consistent test criteria.

The specification problem has been shifted to a higher level but not eliminated.

Testing Techniques for functional bugs: Most functional test techniques- that is those techniques which are based on a behavioural description of software, such as transaction flow testing, syntax testing, domain testing, logic testing and state testing are useful in testing functional bugs.

II. Structural bugs

Various categories in Structural bugs include:

1. Control and Sequence Bugs:

- Control and sequence bugs include paths left out, unreachable code, improper nesting of loops, loop-back or loop termination criteria incorrect, missing process steps, duplicated processing, unnecessary processing, rampaging, GOTO's, ill-conceived (not properly planned) switches, spaghetti code, and worst of all, pachinko code.
- One reason for control flow bugs is that this area is amenable (supportive) to theoretical treatment. Most of the control flow bugs are easily tested and caught in unit testing.
- Another reason for control flow bugs is that use of old code especially ALP & COBOL code are dominated by control flow bugs.
- Control and sequence bugs at all levels are caught by testing, especially structural testing, more specifically path testing combined with a bottom-line functional test based on a specification.

2. Logic Bugs

- Bugs in logic, especially those related to misunderstanding how case statements and logic operators behave singly and combinations
- Also includes evaluation of boolean expressions in deeply nested IF-THEN-ELSE constructs.
- If the bugs are parts of logical (i.e. boolean) processing not related to control flow, they are characterized as processing bugs.
- If the bugs are parts of a logical expression (i.e. control-flow statement) which is used to direct the control flow, then they are categorized as control-flow bugs.

3. Processing Bugs

Processing bugs include arithmetic bugs, algebraic, mathematical function evaluation, algorithm selection and general processing.

Examples of Processing bugs include: Incorrect conversion from one data representation to other, ignoring overflow, improper use of greater-than-or-equal etc. Although these bugs are frequent (12%), they tend to be caught in good unit testing.

4. Initialization Bugs

- Initialization bugs are common. Initialization bugs can be improper and superfluous. Superfluous bugs are generally less harmful but can affect performance.
- Typical initialization bugs include: Forgetting to initialize the variables before first use, assuming that they are initialized elsewhere, initializing to the wrong format, representation. Explicit declaration of all variables, as in Pascal, can reduce some initialization problems.

5. Data-Flow Bugs and Anomalies

- Most initialization bugs are special case of data flow anomalies.
- data flow anomaly occurs where there is a path along which we expect to do something.
- unreasonable with data, such as using an uninitialized variable, attempting to use a variable before it exists, modifying and then not storing or using the result, or initializing twice. Without an intermediate use

III. Data bugs

- Data bugs include all bugs that arise from the specification of data objects, their formats, the number of such objects, and their initial values.
- Data Bugs are at least as common as bugs in code, but they are often treated as if they did not exist at all.
- Code migrates data: Software is evolving towards programs in which more and more of the control and processing functions are stored in tables.
- Because of this, there is an increasing awareness that bugs in code are only half the battle and the data problems should be given equal attention.

Dynamic Data Vs Static data:

Dynamic data are transitory. Whatever their purpose their lifetime is relatively short, typically the processing time of one transaction. A storage object may be used to hold dynamic data of different types, with different formats, attributes and residues.

Dynamic data bugs are due to leftover garbage in a shared resource. This can be handled in one of the three ways: (1) Clean up after the use by the user (2) Common Cleanup by the resource manager (3) No Clean up

Static Data are fixed in form and content. They appear in the source code or database directly or indirectly, for example a number, a string of characters, or a bit pattern. Compile time processing will solve the bugs caused by static data.

Information, parameter, and control:

Static or dynamic data can serve in one of three roles, or in combination of roles: as a parameter, for control, or for information.

Content, Structure and Attributes:

Content can be an actual bit pattern, character string, or number put into a data structure. Content is a pure bit pattern and has no meaning unless it is interpreted by a hardware or software processor. All data bugs result in the corruption or misinterpretation of content

Structure relates to the size, shape and numbers that describe the data object, which is memory location used to store the content. (E.g. A two-dimensional array).

Attributes relates to the specification meaning that is the semantics associated with The contents of a data object. (E.g. an integer, an alphanumeric string, a subroutine). The severity and subtlety of bugs increases as we go from content to attributes because the things get less formal in that direction.

IV. Coding bugs

- Coding errors of all kinds can create any of the other kind of bugs.
- Syntax errors are generally not important in the scheme of things if the source language translator has adequate syntax checking.
- If a program has many syntax errors, then we should expect many logic and coding bugs.
- The documentation bugs are also considered as coding bugs which may mislead the maintenance programmers.

V. Interface, integration, and system bugs

Various categories of bugs in Interface, Integration, and System Bugs are:

A. Interface bugs

1. External Interfaces:

- The external interfaces are the means used to communicate with the world.
- These include devices, actuators, sensors, input terminals, printers, and communication lines.
- The primary design criterion for an interface with outside world should be robustness.
- All external interfaces, human or machine should employ a protocol. The protocol may be wrong or incorrectly implemented. Other external interface bugs are: invalid timing or sequence assumptions related to external signals
- Misunderstanding external input or output formats. Insufficient tolerance to bad input data.

2. Internal Interfaces:

- Internal interfaces are in principle not different from external interfaces but they are more controlled. A best example for internal interfaces is communicating routines.
- The external environment is fixed and the system must adapt to it but the internal environment, which consists of interfaces with other components, can be negotiated. Internal interfaces have the same problem as external interfaces

3. Hardware Architecture:

- Bugs related to hardware architecture originate mostly from misunderstanding how the hardware works.
- Examples of hardware architecture bugs: address generation error, i/o device operation Instruction error, waiting too long for a response, incorrect interrupt handling etc.
- The remedy for hardware architecture and interface problems is twofold: (1) Good Programming and Testing (2) Centralization of hardware interface software in Programs written by hardware interface specialists

4. Operating System Bugs:

- Program bugs related to the operating system are a combination of hardware architecture and interface bugs mostly caused by a misunderstanding of what it is the operating system does.
- Use operating system interface specialists, and use explicit interface modules or macros for all operating system calls.
- This approach may not eliminate the bugs but at least will localize them and make testing easier.

5. Software Architecture:

- Software architecture bugs are the kind that called - interactive.
- Routines can pass unit and integration testing without revealing such bugs.
- Many of them depend on load, and their symptoms emerge only when the system is stressed. Sample for such bugs: Assumption that there will be no interrupts, Failure to block or unblock interrupts, Assumption that memory and registers were initialized or not initialized Careful integration of modules and subjecting the final system to a stress test are effective methods for these bugs.

6. Control and Sequence Bugs (Systems Level):

- These bugs include: Ignored timing, Assuming that events occur in a specified sequence, Working on data before all the data have arrived from disc, Waiting for an impossible combination of prerequisites, Missing, wrong, redundant or superfluous process steps.
- The remedy for these bugs is highly structured sequence control. Specialize, internal, sequence control mechanisms are helpful

7. Resource Management Problems

- Memory is subdivided into dynamically allocated resources such as buffer blocks, queue blocks, task control blocks, and overlay buffers.
- External mass storage units such as discs, are subdivided into memory resource pools.
- Some resource management and usage bugs: Required resource not obtained, Wrong resource used, Resource is already in use, Resource dead lock etc

Resource Management Remedies

A design remedy that prevents bugs is always preferable to a test method that discovers them. The design remedy in resource management is to keep the resource structure simple: the fewest different kinds of resources, the fewest pools, and no private resource management.

B. Integration Bugs

- Integration bugs are bugs having to do with the integration of, and with the interfaces between, working and tested components.
- These bugs result from inconsistencies or incompatibilities between components.
- The communication methods include data structures, call sequences, registers, semaphores, and communication links and protocols result in integration bugs.
- The integration bugs do not constitute a big bug category (9%) they are expensive category because they are usually caught late in the game and because they force changes in several components and/or data structures.

C. System Bugs

- System bugs covering all kinds of bugs that cannot be ascribed to a component or to their simple interactions, but result from the totality of interactions between many components such as programs, data, hardware, and the operating systems.
- There can be no meaningful system testing until there has been thorough component and integration testing. System bugs are infrequent (1.7%) but very important because they are often found only after the system has been fielded.

VI. TEST AND TEST DESIGN BUGS

- Testing: testers have no immunity to bugs. Tests require complicated scenarios and databases. They require code or the equivalent to execute and consequently they can have bugs.
- Test criteria: if the specification is correct, it is correctly interpreted and implemented, and a proper test has been designed; but the criterion by which the software's behavior is judged may be incorrect or impossible. So, a proper test criterion has to be designed.
- The more complicated the criteria, the likelier they are to have bugs.

Remedies: The remedies of test bugs are:

1. **Test Debugging:** The first remedy for test bugs is testing and debugging the tests. Test debugging, when compared to program debugging, is easier because tests, when properly designed are simpler than programs and do not have to make concessions to efficiency.
2. **Test Quality Assurance:** Programmers have the right to ask how quality in independent testing is monitored.
3. **Test Execution Automation:** The history of software bug removal and prevention is indistinguishable from the history of programming automation aids. Assemblers, loaders, compilers are developed to reduce the incidence of programming and operation errors. Test execution bugs are virtually eliminated by various test execution automation tools.
4. **Test Design Automation:** Just as much of software development has been automated, much test design can be and has been automated. For a given productivity rate, automation reduces the bug count - be it for software or be it for tests.

FLOW GRAPHS AND PATH TESTING

BASICS OF PATH

Testing:

- Path Testing is the name given to a family of test techniques based on judiciously selecting a set of test paths through the program. o If the set of paths are properly chosen then we have achieved some measure of test thoroughness. For example, pick enough paths to assure that every source statement has been executed at least once.
- Path testing techniques are the oldest of all structural test techniques.
- Path testing is most applicable to new software for unit testing. It is a structural technique.
- It requires complete knowledge of the program's structure. o It is most often used by programmers to unit test their own code.
- The effectiveness of path testing rapidly deteriorates as the size of the software aggregate under test increases.

The Bug Assumption:

- The bug assumption for the path testing strategies is that something has gone wrong with the software that makes it take a different path than intended.
- As an example "GOTO X" where "GOTO Y" had been intended.
- Structured programming languages prevent many of the bugs targeted by path testing: as a consequence, the effectiveness for path testing for these languages is reduced and for old code in COBOL, ALP, FORTRAN and Basic, the path testing is indispensable.

Control Flow Graphs:

- The control flow graph is a graphical representation of a program's control structure. It uses the elements named process blocks, decisions, and junctions. The flow graph is similar to the earlier flowchart, with which it is not to be confused.
- **Flow Graph Elements:** A flow graph contains four different types of elements. Process Block (2) Decisions (3) Junctions (4) Case Statements.

1.Process Block:

process block is a sequence of program statements uninterrupted by either decisions or junctions. It is a sequence of statements such that if any one of statement of the block is executed, then all statement thereof are executed. Formally, a process block is a piece of straight-line code of one statement or hundreds of statements.

process has one entry and one exit. It can consist of a single statement or instruction, a sequence of statements or instructions, a single entry/exit subroutine, a macro or function.

2. Decisions: A decision is a program point at which the control flow can diverge. Machine language conditional branch and conditional skip instructions are examples of decisions. Most of the decisions are two-way but some are three-way branches in control flow.

3. Junctions:

A junction is a point in the program where the control flow can merge.

Examples of junctions are: the target of a jump or skip instruction in ALP, a label that is a target of GOTO.

4. Case Statements: A case statement is a multi-way branch or decisions. Examples of case statement are a jump table in assembly language, and the PASCAL case statement. From the point of view of test design, there are no differences between Decisions and Case Statements.

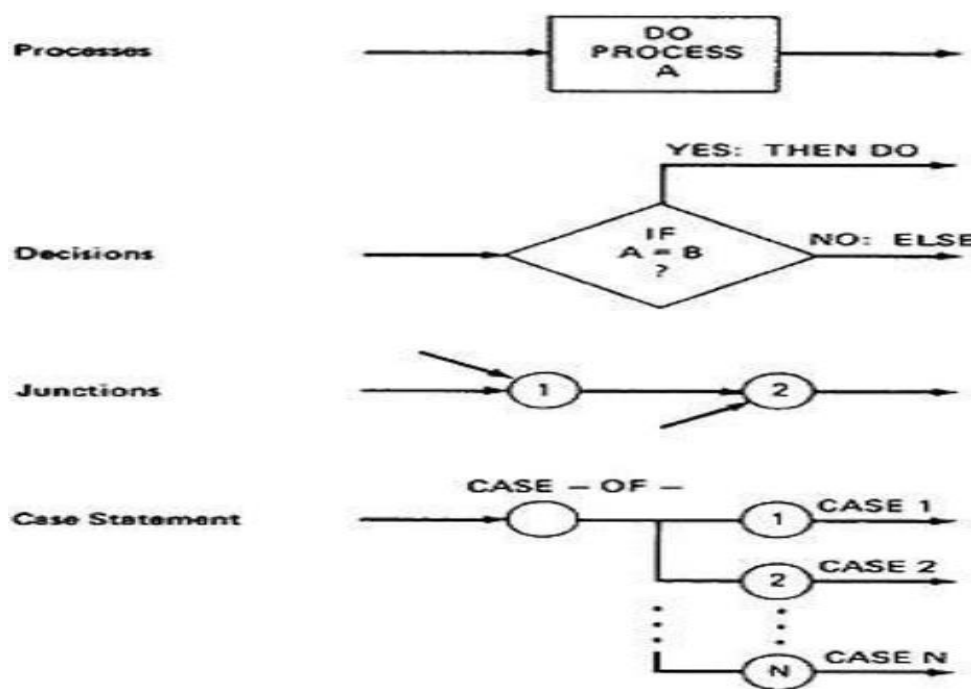


Figure 2.1: Flow graph Elements

Control Flow Graphs Vs Flowcharts:

- A program's flow chart resembles a control flow graph.
- In flow graphs, we don't show the details of what is in a process block. In flowcharts every part of the process block is drawn. The flowchart focuses on process steps, where as the flow graph focuses on control flow of the program.
- The act of drawing a control flow graph is a useful tool that can help us clarify the control flow and data flow issues.

Notational Evolution:

- The control flow graph is simplified representation of the program's structure. The notation changes made in creation of control flow graphs:
- The process boxes weren't really needed. There is an implied process on every line joining junctions and decisions.
- We don't need to know the specifics of the decisions, just the fact that there is a branch.
- The specific target label names aren't important-just the fact that they exist. So we can replace them by simple numbers. To understand this, we will go through an example (Figure 2.2) written in a FORTRAN like programming language called Programming Design Language (PDL).
- The program's corresponding flowchart (Figure 2.3) and flowgraph (Figure 2.4) were also provided below for better understanding. o The first step in translating the program to a flowchart is shown in Figure 2.3, where we have the typical one-for-one classical flowchart. Note that complexity has increased, clarity has decreased, and that we had to add auxiliary labels (LOOP, XX, and YY), which have no actual program counterpart. In Figure 2.4 we merged the process steps and replaced them with the single process box.
- We now have a control flow graph. But this representation is still too busy. We simplify the notation further to achieve Figure 2.5, where for the first time we can really see what the control flow looks like.

```
CODE* (PDL)

INPUT X, Y
Z := X + Y
V := X - Y
IF Z >= 0 GOTO SAM
JOE: Z := Z - 1
SAM: Z := Z + V
FOR U = 0 TO Z
  V(U), U(V) := (Z + V)*U
  IF V(U) = 0 GOTO JOE
  Z := Z - 1
  IF Z = 0 GOTO ELL
  U := U + 1
NEXT U

V(U-1) := V(U+1) + U(V-1)
ELL: V(U+U(V)) := U + V
IF U = V GOTO JOE
IF U > V THEN U := Z
Z := U
END
```

* A contrived horror

Figure 2.2: Program Example (PDL)

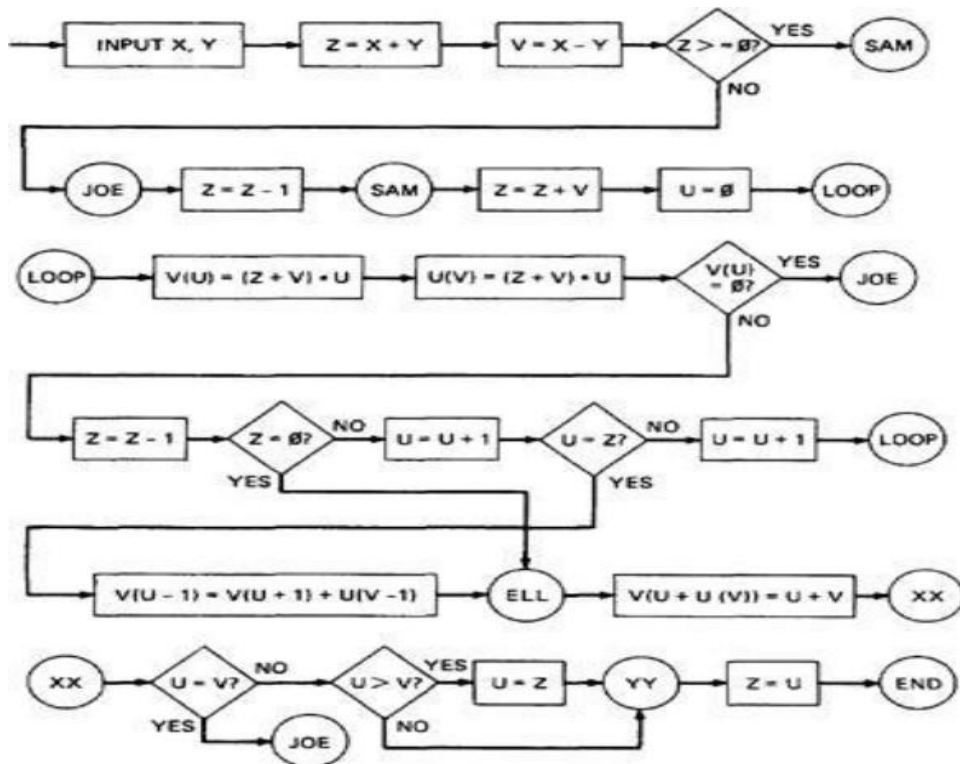


Figure 2.3: One-to-one flowchart for example program in Figure 2.

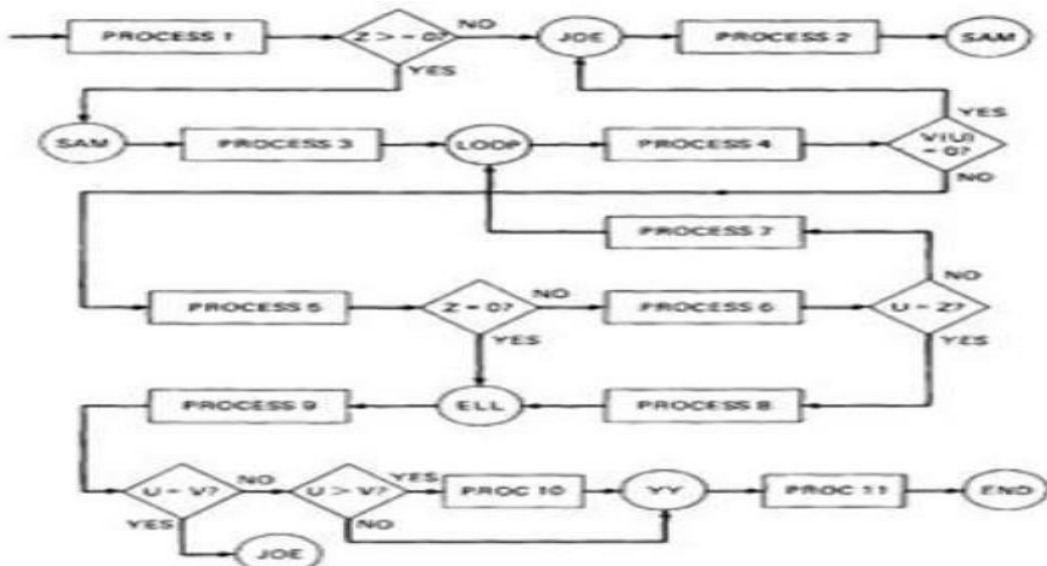


Figure 2.4: Control Flow graph for example in Figure 2.2

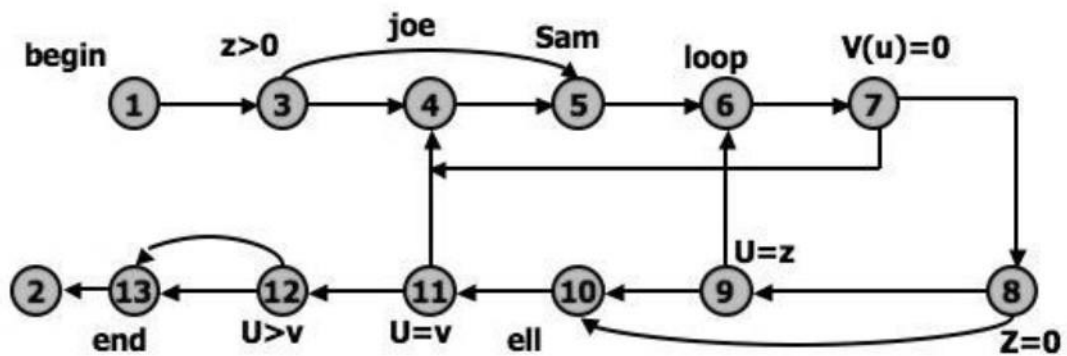


Figure 2.5: Simplified Flow graph Notation

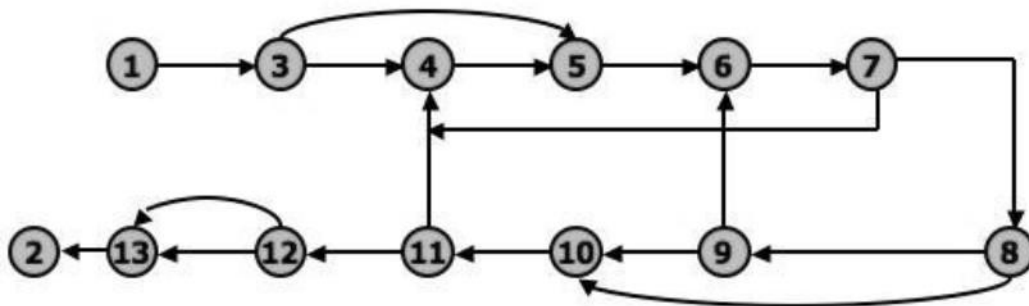


Figure 2.6: Even Simplified Flow graph Notation

The final transformation is shown in Figure 2.6, where we've dropped the node numbers to achieve an even simpler representation. The way to work with control flow graphs is to use the simplest possible representation that is, no more information than you need to correlate back to the source program or PDL.

Linked list representation:

Although graphical representations of flow graphs are revealing, the details of the control flow inside a program they are often inconvenient.

In linked list representation, each node has a name and there is an entry on the list for each link in the flow graph. Only the information pertinent to the control flow is shown.

Linked List representation of Flow Graph

1 (BEGIN)	:	3	
2 (END)	:		Exit, no outlink
3 (Z>0?)	:	4 (FALSE)	
	:	5 (TRUE)	
4 (JOE)	:	5	
5 (SAM)	:	6	
6 (LOOP)	:	7	
7 (V(U)=0?)	:	4 (TRUE)	
	:	8 (FALSE)	
8 (Z=0?)	:	9 (FALSE)	
	:	10 (TRUE)	
9 (U=Z?)	:	6 (FALSE) = LOOP	
	:	10 (TRUE) = ELL	
10 (ELL)	:	11	
11 (U=V?)	:	4 (TRUE) = JOE	
	:	12 (FALSE)	
12 (U>V?)	:	13 (TRUE)	
	:	13 (FALSE)	
13	:	2 (END)	

Figure 2.7: Linked List Control Flow graph Notation

Flowgraph - program correspondence

A flow graph is a pictorial representation of a program and not the program itself, just as a topographic map. You can't always associate the parts of a program in a unique way with flow graph parts because many program structures, such as if-then-else constructs, consists of a combination of decisions, junctions, and processes. The translation from a flow graph element to a statement and vice versa is not always unique. (See Figure 2.8)

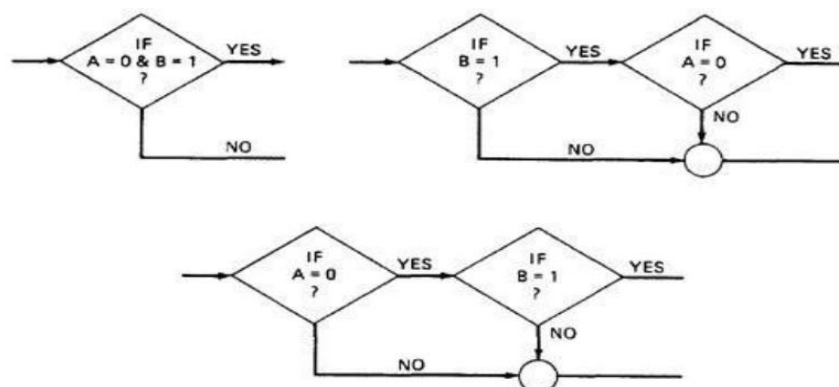


Figure 2.8: Alternative Flow graphs for same logic
(Statement "IF (A=0) AND (B=1) THEN ...").

An improper translation from flow graph to code during coding can lead to bugs, and improper translation during the test design lead to missing test cases and causes undiscovered bugs.

Flowgraph and Flowchart generation

Flowcharts can be

1. Handwritten by the programmer.
2. Automatically produced by a flowcharting program based on a mechanical analysis of the source code.
3. Semi automatically produced by a flow charting program based in part on structural analysis of the source code and in part on directions given by the programmer.

There are relatively few control flow graph generators.

PATH TESTING - PATHS, NODES AND LINKS

Path: A path through a program is a sequence of instructions or statements that starts at an entry, junction, or decision and ends at another, or possibly the same junction, decision, or exit.

- A path may go through several junctions, processes, or decisions, one or more times.
- Paths consist of segments. The segment is a link - a single process that lies between two nodes. A path segment is succession of consecutive links that belongs to some path.
- The length of path measured by the number of links in it and not by the number of the instructions or statements executed along that path. The name of a path is the name of the nodes along the path.

Fundamental path selection criteria

There are many paths between the entry and exit of a typical routine.

Every decision double the number of potential paths. And every loop multiplies the number of potential paths by the number of different iteration values possible for the loop. Defining complete testing:

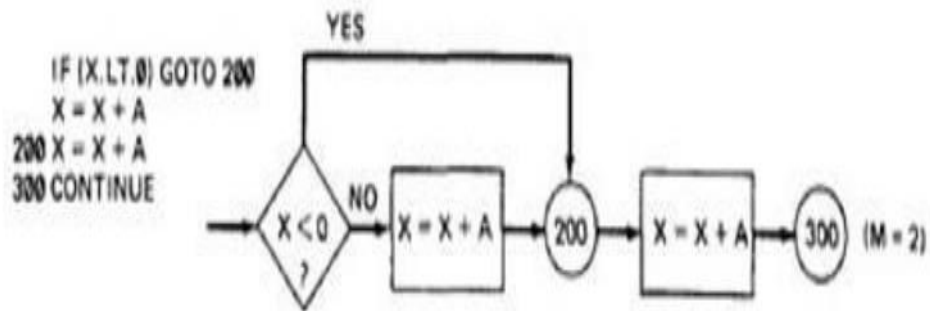
Exercise every path from entry to exit.

Exercise every statement or instruction at least once.

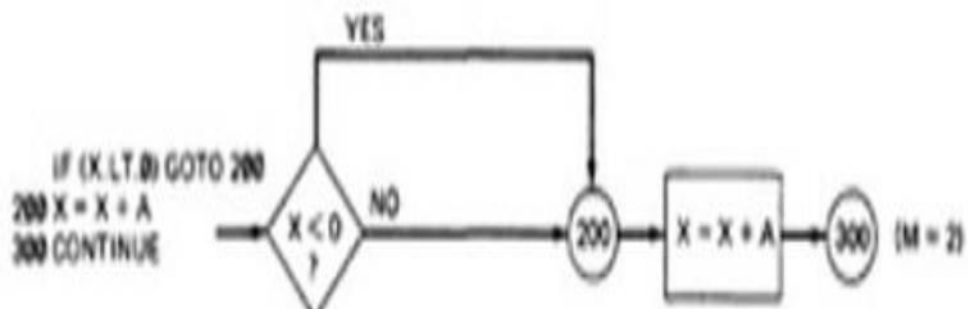
Exercise every branch and case statement, in each direction at least once.

If prescription 1 is followed then 2 and 3 are automatically followed. But it is impractical for most routines. It can be done for the routines that have no loops, in which it is equivalent to 2 and 3 prescriptions.

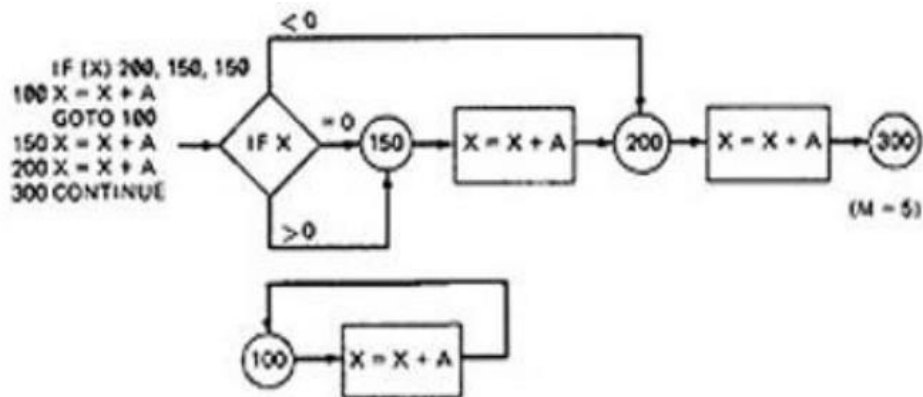
EXAMPLE: Here is the correct version.



For X negative, the output is $X + A$, while for X greater than or equal to zero, the output is $X + 2A$. Following prescription 2 and executing every statement, but not every branch, would not reveal the bug in the following incorrect version:



A negative value produces the correct answer. Every statement can be executed, but if the test cases do not force each branch to be taken, the bug can remain hidden. The next example uses a test based on executing each branch but does not force the execution of all statements:



The hidden loop around label 100 is not revealed by tests based on prescription 3 alone because no test forces the execution of statement 100 and the following GOTO statement. Furthermore, label 100 is not flagged by the compiler as an unreferenced label and the subsequent GOTO does not refer to an undefined label.

A **Static Analysis** (that is, an analysis based on examining the source code or structure) cannot determine whether a piece of code is or is not reachable. There could be subroutine calls with parameters that are subroutine labels, or in the above example there could be a GOTO that targeted label 100 but could never achieve a value that would send the program to that label.

A **Dynamic Analysis** (that is, an analysis based on the code's behavior while running - which is to say, to all intents and purposes, testing) can determine whether code is reachable or not and therefore distinguish between the ideal structure we think we have and the actual, buggy structure.

PATH TESTING CRITERIA:

- Any testing strategy based on paths must at least both exercise every instruction and take branches in all directions.
- A set of tests that does this is not complete in an absolute sense, but it is complete in the sense that anything less must leave something untested.
- So we have explored three different testing criteria or strategies out of a potentially infinite family of strategies.
 - i. **Path Testing (P_{inf}):**
 1. Execute all possible control flow paths through the program: typically, this is restricted to all possible entry/exit paths through the program.
 2. If we achieve this prescription, we are said to have achieved 100% path coverage. This is the strongest criterion in the path testing strategy family: it is generally impossible to achieve.

ii. **Statement Testing (P_1):**

1. Execute all statements in the program at least once under some test. If we do enough tests to achieve this, we are said to have achieved 100% statement coverage.
2. An alternate equivalent characterization is to say that we have achieved 100% node coverage. We denote this by C1.
3. This is the weakest criterion in the family: testing less than this for new software is unconscionable (unprincipled or cannot be accepted) and should be criminalized.

iii. **Branch Testing (P_2):**

1. Execute enough tests to assure that every branch alternative has been exercised at least once under some test.
2. If we do enough tests to achieve this prescription, then we have achieved 100% branch coverage.
3. An alternative characterization is to say that we have achieved 100% link coverage.
4. For structured software, branch testing and therefore branch coverage strictly includes statement coverage.
5. We denote branch coverage by C2.

Commonsense and Strategies:

- Branch and statement coverage are accepted today as the minimum mandatory testing requirement.
- The question "why not use a judicious sampling of paths?, what is wrong with leaving some code, untested?" is ineffectual in the view of common sense and experience since: **(1.)** Not testing a piece of a code leaves a residue of bugs in the program in proportion to the size of the untested code and the probability of bugs. **(2.)** The high probability paths are always thoroughly tested if only to demonstrate that the system works properly.
- **Which paths to be tested?** You must pick enough paths to achieve C1+C2. The question of what is the fewest number of such paths is interesting to the designer of test tools that help automate the path testing, but it is not crucial to the pragmatic (practical) design of tests. It is better to make many simple paths than a few complicated paths.

Path Selection Example:

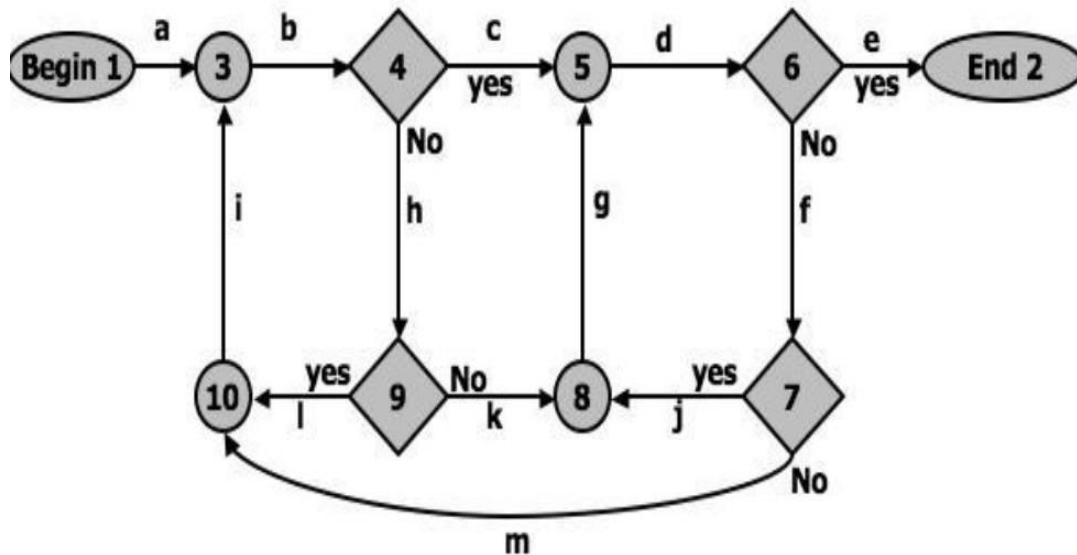


Figure 2.9: An example flow graph to explain path selection

Practical Suggestions in Path Testing:

1. Draw the control flow graph on a single sheet of paper.
2. Make several copies - as many as you will need for coverage (C1+C2) and several more.
3. Use a yellow highlighting marker to trace paths. Copy the paths onto master sheets.
4. Continue tracing paths until all lines on the master sheet are covered, indicating that you appear to have achieved C1+C2.
5. As you trace the paths, create a table that shows the paths, the coverage status of each process, and each decision.
6. The above paths lead to the following table considering Figure 2.9

PATHS	DECISIONS				PROCESS—LINK												
	4	6	7	9	a	b	c	d	e	f	g	h	i	j	k	l	m
abcde	YES	YES			✓	✓	✓	✓	✓								
abhkgde	NO	YES		NO	✓	✓		✓	✓		✓	✓			✓		
abhlibcde	NO,YES	YES		YES	✓	✓	✓	✓	✓			✓	✓			✓	
abcdfjgde	YES	NO,YES	YES		✓	✓	✓	✓	✓	✓	✓			✓			
abcdfmibcde	YES	NO,YES	NO		✓	✓	✓	✓	✓	✓			✓				✓

7. After you have traced a covering path set on the master sheet and filled in the table for every path, check the following:

1. Does every decision have a YES and a NO in its column? (C2)
2. Has every case of all case statements been marked?(C2)
3. Is every three - way branch (less, equal, greater) covered? (C2)

Is every link (process) covered at least once? (C1)

8. Revised Path Selection Rules: β Pick the simplest, functionally sensible entry/exit path. β Pick additional paths as small variation from previous paths. Pick paths that do not have loops rather than paths that do. Favor short paths that make sense over paths that don't. Pick additional paths that have no obvious functional meaning only if it's necessary to provide coverage. Be comfortable with your chosen paths. Play your hunches (guesses) and give your intuition free reign as long as you achieve C1+C2. Don't follow rules slavishly (blindly) - except for coverage.

LOOPS:

Cases for a single loop: A Single loop can be covered with two cases: Looping and Not looping. But, experience shows that many loop-related bugs are not discovered by C1+C2. Bugs hide themselves in corners and congregate at boundaries - in the cases of loops, at or around the minimum or maximum number of times the loop can be iterated. The minimum number of iterations is often zero, but it need not be.

CASE 1: Single loop, Zero minimum, N maximum, No excluded values

1. Try bypassing the loop (zero iterations). If you can't, you either have a bug, or zero is not the minimum and you have the wrong case.
2. Could the loop-control variable be negative? Could it appear to specify a negative number of iterations? What happens to such a value?
3. One pass through the loop.
4. Two passes through the loop.
5. A typical number of iterations, unless covered by a previous test.
6. One less than the maximum number of iterations.
7. The maximum number of iterations.
8. Attempt one more than the maximum number of iterations. What prevents the loop control variable from having this value? What will happen with this value if it is forced?

CASE 2: Single loop, Non-zero minimum, No excluded values

1. Try one less than the expected minimum. What happens if the loop control variable's value is less than the minimum? What prevents the value from being less than the minimum?
2. The minimum number of iterations.
3. One more than the minimum number of iterations.
4. Once, unless covered by a previous test.
5. Twice, unless covered by a previous test.
6. A typical value.
7. One less than the maximum value.
8. The maximum number of iterations.
9. Attempt one more than the maximum number of iterations.

CASE 3: Single loops with excluded values

- Treat single loops with excluded values as two sets of tests consisting of loops without excluded values, such as case 1 and 2 above.
- Example, the total range of the loop control variable was 1 to 20, but that values 7, 8, 9, 10 were excluded. The two sets of tests are 1-6 and 11-20.
- The test cases to attempt would be 0, 1, 2, 4, 6, 7 for the first range and 10, 11, 15, 19, 20, 21 for the second range.

Nested Loops:

The number of tests to be performed on nested loops will be the exponent of the tests performed on single loops. As we cannot always afford to test all combinations of nested loops' iterations values. Here's a tactic used to discard some of these values:

1. Start at the inner most loop. Set all the outer loops to their minimum values.
2. Test the minimum, minimum+1, typical, maximum-1 , and maximum for the innermost loop, while holding the outer loops at their minimum iteration parameter values. Expand the tests as required for out of range and excluded values.
3. If you've done the outmost loop, GOTO step 5, else move out one loop and set it up as in step 2 with all other loops set to typical values.
4. Continue outward in this manner until all loops have been covered.
5. Do all the cases for all loops in the nest simultaneously.

Concatenated Loops:

Concatenated loops fall between single and nested loops with respect to test cases. Two loops are concatenated if it's possible to reach one after exiting the other while still on a path from entrance to exit. If the loops cannot be on the same path, then they are not concatenated and can be treated as individual loops.

Horrible Loops:

A horrible loop is a combination of nested loops, the use of code that jumps into and out of loops, intersecting loops, hidden loops, and cross connected loops.

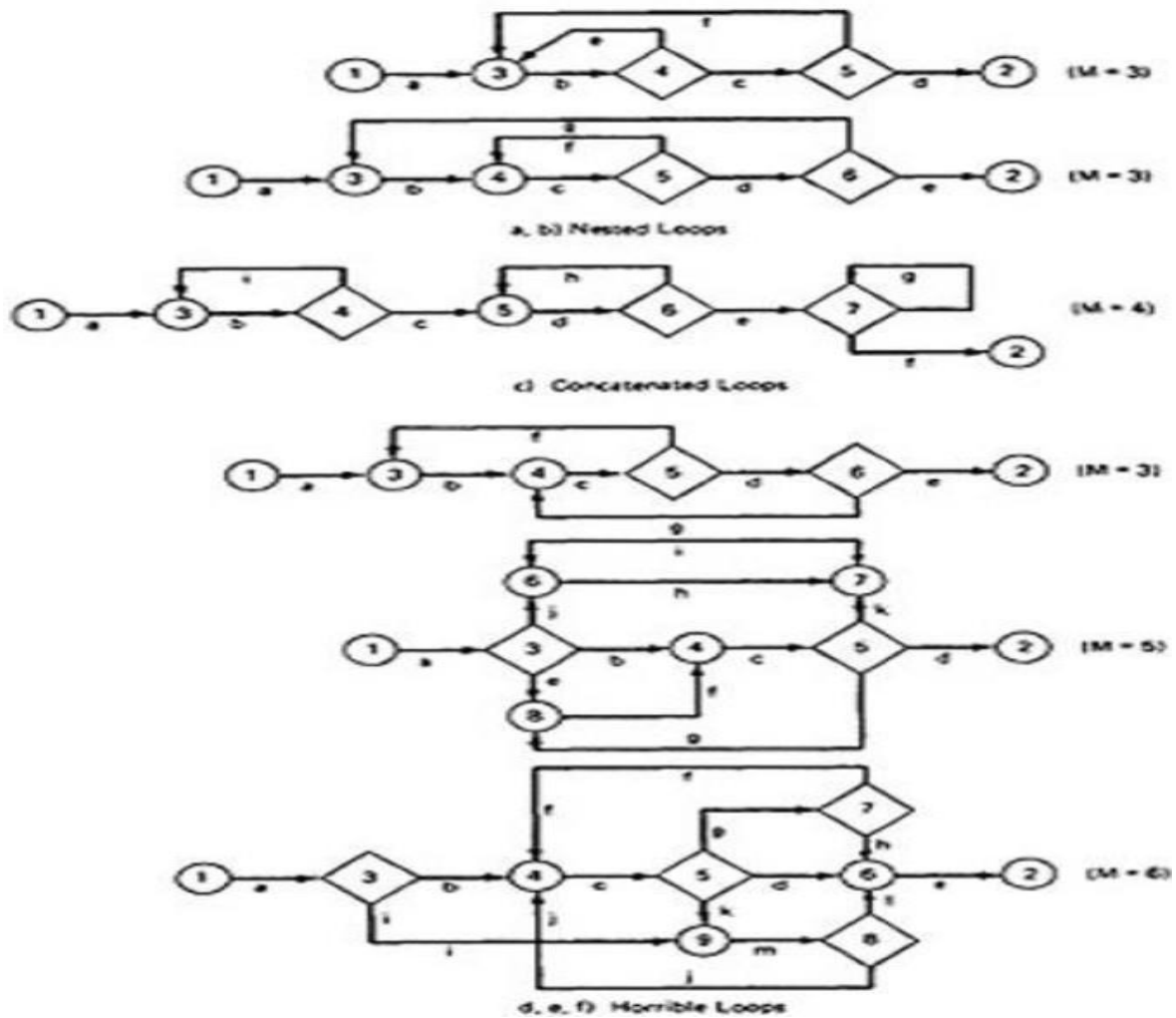
Makes iteration value selection for test cases an awesome and ugly task, which is another reason such structures should be avoided.

Loop Testing Time:

Any kind of loop can lead to long testing time, especially if all the extreme value cases are to attempted (Max-1, Max, Max+1).

- This situation is obviously worse for nested and dependent concatenated loops.
- Consider nested loops in which testing the combination of extreme values lead to long test times. Several options to deal with:
- Prove that the combined extreme cases are hypothetically possible, they are not possible in the real world
- Put in limits or checks that prevent the combined extreme cases. Then you have to test the software that implements such safety measures.

Figure 2.10: Example of Loop types



PREDICATES, PATH PREDICATES AND ACHIEVABLE PATHS

PREDICATE: The logical function evaluated at a decision is called Predicate. The direction taken at a decision depends on the value of decision variable. Some examples are: $A > 0$, $x + y \geq 90$

PATH PREDICATE: A predicate associated with a path is called a Path Predicate. For example, "x is greater than zero", " $x + y \geq 90$ ", "w is either negative or equal to 10 is true" is a sequence of predicates whose truth values will cause the routine to take a specific path.

Multiway branches

The path taken through a multiway branch such as a computed GOTO's, case statement, or jump tables cannot be directly expressed in TRUE/FALSE terms.

Although, it is possible to describe such alternatives by using multi valued logic, an expedient (practical approach) is to express multiway branches as an equivalent set of if..then..else statements.

For example, a three-way case statement can be written as: If case=1 DO A1 ELSE (IF Case=2 DO A2 ELSE DO A3 ENDIF) ENDIF.

INPUTS

In testing, the word input is not restricted to direct inputs, such as variables in a subroutine call, but includes all data objects referenced by the routine whose values are fixed prior to entering it.

For example, inputs in a calling sequence, objects in a data structure, values left in registers, or any combination of object types.

The input for a particular test is mapped as a one-dimensional array called as an Input Vector.

Predicate interpretation

- The simplest predicate depends only on input variables.
- For example if x_1, x_2 are inputs, the predicate might be $x_1 + x_2 \geq 7$, given the values of x_1 and x_2 the direction taken through the decision is based on the predicate is determined at input time and does not depend on processing.
- Another example, assume a predicate $x_1 + y \geq 0$ that along a path prior to reaching this predicate we had the assignment statement $y = x_2 + 7$. although our predicate depends on processing, we can substitute the symbolic expression for y to obtain an equivalent predicate $x_1 + x_2 + 7 \geq 0$.
- The act of symbolic substitution of operations along the path in order to express the predicate solely in terms of the input vector is called **predicate interpretation**.

Sometimes the interpretation may depend on the path; for example

```
INPUT X
ON X GOTO A, B, C, ...
A: Z := 7 @
GOTO
HEM B: Z
:= 7 @
GOTO
HEM C: Z
:= 0 @
GOTO HEM
.....
```

HEM: DO SOMETHING

.....

HEN: IF $Y + Z > 0$ GOTO ELL ELSE GOTO EMM

The predicate interpretation at HEN depends on the path we took through the first multiway branch. It yields for the three cases respectively, if $Y+7>0$, $Y-7>0$, $Y>0$.

The path predicates are the specific form of the predicates of the decisions along the selected path after interpretation.

Independence of variables and predicates

- The path predicates take on truth values based on the values of input variables, either directly or indirectly.
- If a variable's value does not change as a result of processing, that variable is independent of the processing.
- If the variable's value can change as a result of the processing, the variable is process dependent.
- A predicate whose truth value can change as a result of the processing is said to be **process dependent** and one whose truth value does not change as a result of the processing is **process independent**.
- Process dependence of a predicate does not always follow from dependence of the input variables on which that predicate is based.

Correlation of variables and predicates

- Two variables are correlated if every combination of their values cannot be independently specified.
- Variables whose values can be specified independently without restriction are
- called uncorrelated. A pair of predicates whose outcomes depend on one or more
- variables in common are said to be correlated predicates.
- For example, the predicate $X=Y$ is followed by another predicate $X+Y = 8$. If we
- select X and Y values to satisfy the first predicate, we might have forced the 2nd
- predicate's truth value to change.
- Every path through a routine is achievable only if all the predicates in that routine are uncorrelated.

PATH PREDICATE EXPRESSIONS:

A path predicate expression is a set of boolean expressions, all of which must be satisfied to achieve the selected path.

Example:

$$X1+3X2+17 \geq 0 \quad X3=17$$

$$X4-X1 \geq 14X2$$

Any set of input values that satisfy all of the conditions of the path predicate expression will force the routine to the path. Sometimes a predicate can have an OR in it.

Example:

$$A: X5 > 0$$

$$B: X1 + 3X2 + 17 \geq 0$$

$$C: X3 = 17$$

$$D: X4 - X1 \geq 14X2$$

$$E: X6 < 0$$

$$B: X1 + 3X2 + 17$$

$$\geq 0$$

$$C: X3 = 17$$

$$D: X4 - X1 \geq 14X2$$

Boolean algebra notation to denote the boolean expression:

$$ABCD + EBCD = (A + E)BCD$$

PREDICATE COVERAGE

Compound Predicate: Predicates of the form A OR B, A AND B and more complicated.

- Boolean expressions are called as compound predicates.
- Sometimes even a simple predicate becomes compound after interpretation.
Example: the predicate if ($x=17$) whose opposite branch is if $x \neq 17$ which is equivalent to $x > 17$. Or. $x < 17$.
- Predicate coverage is being the achieving of all possible combinations of truth values corresponding to the selected path have been explored under some test.
- As achieving the desired direction at a given decision could still hide bugs in the associated predicates.

Testing blindness

Testing Blindness is a pathological (harmful) situation in which the desired path is achieved for the wrong reason.

There are three types of Testing Blindness

Assignment Blindness:

Assignment blindness occurs when the buggy predicate appears to work correctly because the specific value chosen for an assignment statement works with both the correct and incorrect predicate.

For Example:

Correct			Buggy		
X	=	7	X	=	7
.....					
if Y > then		0	if X+Y > then		0
...			...		

- ? If the test case sets Y=1 the desired path is taken in either case, but there is still a bug.

Equality Blindness:

Equality blindness occurs when the path selected by a prior predicate result in a value that works both for the correct and buggy predicate.

For Example:

Correct	Buggy
if Y = 2 then	if Y = 2 then
.....	
if X+Y > 3	if X > 1
then ...	then ...

The first predicate if y=2 forces the rest of the path, so that for any positive value of x. the path taken at the second predicate will be the same for the correct and buggy version.

Self-Blindness:

Self-blindness occurs when the buggy predicate is a multiple of the correct predicate and as a result is indistinguishable along that path.

For Example:

Correct	Buggy
$X = A$ if $X-1 > 0$ then ...	$X = A$ if $X+A-2 > 0$ then ...

1. The assignment ($x=a$) makes the predicates multiples of each other, so the direction taken is the same for the correct and buggy version.

PATH SENSITIZING

Review: achievable and unachievable paths:

1. We want to select and test enough paths to achieve a satisfactory notion of test completeness such as $C1+C2$.
2. Extract the programs control flow graph and select a set of tentative covering paths.
3. For any path in that set, interpret the predicates along the path as needed to express them in terms of the input vector. In general individual predicates are compound or may become compound as a result of interpretation.
4. Trace the path through, multiplying the individual compound predicates to achieve a boolean expression such as

$$(A+BC) (D+E) (FGH) (IJ) (K) (l) (L).$$

5. Multiply out the expression to achieve a sum of products form:
 $ADFGHIJKL+AEFGHIJKL+BCDFGHIJKL+BCEFGHIJKL$
6. Each product term denotes a set of inequalities that if solved will yield an input vector that will drive the routine along the designated path.
7. Solve any one of the inequality sets for the chosen path and you have found a set of input values for the path.
8. If you can find a solution, then the path is achievable.
9. If you can't find a solution to any of the sets of inequalities, the path is unachievable.
10. The act of finding a set of solutions to the path predicate expression is called **path sensitization**.

Heuristic procedures for sensitizing paths

1. This is a workable approach, instead of selecting the paths without considering how to sensitize, attempt to choose a covering path set that is easy to sensitize and pick hard to sensitize paths only as you must to achieve coverage.
2. Identify all variables that affect the decision.
3. Classify the predicates as dependent or independent.
4. Start the path selection with uncorrelated, independent predicates.
5. If coverage has not been achieved using independent uncorrelated predicates, extend the path set using correlated predicates.
6. If coverage has not been achieved extend the cases to those that involve dependent predicates.
7. Last, use correlated, dependent predicates.

PATH INSTRUMENTATION

1. Path instrumentation is what we have to do to confirm that the outcome was achieved by the intended path.
2. **Co-incident Correctness** The coincidental correctness stands for achieving the desired outcome for wrong reason.

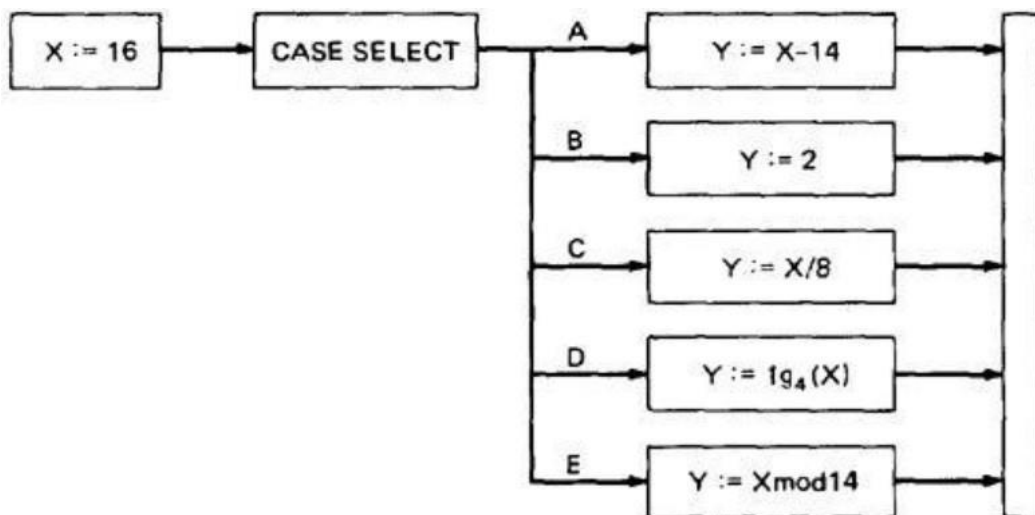


Figure 2.11: Coincidental Correctness

The above figure is an example of a routine that, for the (unfortunately) chosen input value ($X = 16$), yields the same outcome ($Y = 2$) no matter which case we select. Therefore, the tests chosen this way will not tell us whether we have achieved coverage. For example, the five cases could be totally jumbled and still the outcome would be the same. **Path Instrumentation** is what we have to do to confirm that the outcome was achieved by the intended path.

The types of instrumentation methods include:

1. Interpretive Trace Program:

- An interpretive trace program is one that executes every statement in order and records the intermediate values of all calculations, the statement labels traversed etc.
- If we run the tested routine under a trace, then we have all the information we need to confirm the outcome and, furthermore, to confirm that it was achieved by the intended path.
- The trouble with traces is that they give us far more information than we need. In fact, the typical trace program provides so much information that confirming the path from its massive output dump is more work than simulating the computer by hand to confirm the path.

2. Traversal Marker or Link Marker:

A simple and effective form of instrumentation is called a traversal marker or link marker. Name every link by a lower case letter. o Instrument the links so that the link's name is recorded when the link is executed.

The succession of letters produced in going from the routine's entry to its exit should, if there are no bugs, exactly correspond to the path name.

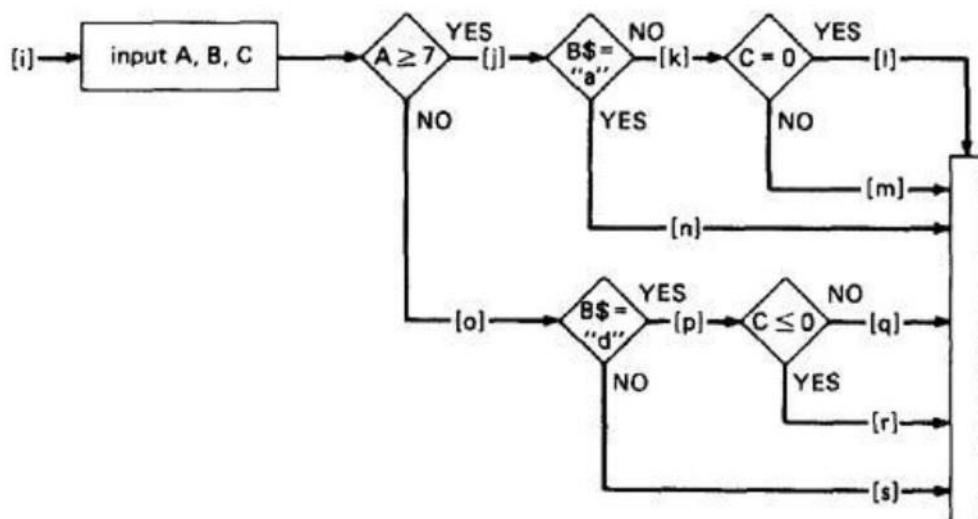


Figure 2.12: Single Link Marker Instrumentation

- **Why Single Link Markers aren't enough:** Unfortunately, a single link marker may not do the trick because links can be chewed by open bugs.

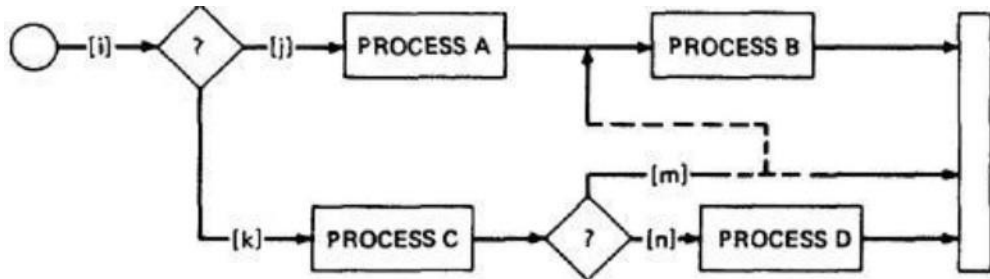


Figure 2.13: Why Single Link Markers aren't enough.

We intended to traverse the i km path, but because of a rampaging GOTO in the middle of the m link, we go to process B. If coincidental correctness is against us, the outcomes will be the same and we won't know about the bug.

Two Link Marker Method:

The solution to the problem of single link marker method is to implement two markers per link: one at the beginning of each link and one at the end. The two link markers now specify the path name and confirm both the beginning and end of the link.

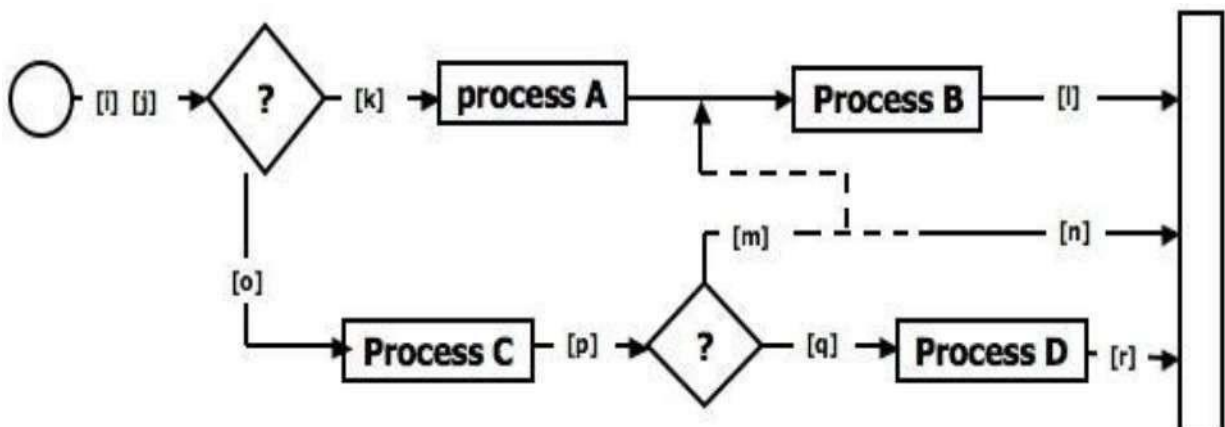


Figure 2.14: Double Link Marker Instrumentation

Link Counter

A less disruptive (and less informative) instrumentation method is based on counters. Instead of a unique link name to be pushed into a string when the link is traversed, we simply increment a link counter. We now confirm that the path length is as expected. The same problem that led us to double link markers also leads us to double link counters.

Applications of Path Testing

Path testing is a **white-box software testing technique** in which different execution paths of a program are tested to ensure the program works correctly under all possible conditions.

Its main applications include:

1. **Finding logical errors**
Detects incorrect program logic, missing conditions, and faulty decision-making.
2. **Ensuring complete code coverage**
Verifies that all independent execution paths in the program are executed at least once.
3. **Testing loops and conditional statements**
Checks the correctness of if, else, switch, for, while, and nested loops.
4. **Detecting dead or unreachable code**
Identifies portions of code that can never be executed.
5. **Validating complex algorithms**
Ensures that algorithms with multiple decision points behave correctly for different inputs.
6. **Improving software reliability**
Reduces the chances of hidden defects by testing various execution paths.
7. **Supporting regression testing**
Confirms that code modifications have not introduced errors into existing execution paths.
8. **Optimizing code quality**
Helps developers identify redundant paths and simplify complex program logic.